ASSIGNMENT 4

1. **Analyze the two GenAI programs. The analysis must include:**
   a. **Name of two GenAI programs**

      Answer: Gemini and claude

   b. **How you accessed them (e.g., Google query, CoPilot on my computer)**

      Answer: Accessed them with their official websites:

      Gemini: https://gemini.google.com/

      Claude: https://claude.ai/

   c. **Prompt used. (You must use the same prompt for both GenAIs.)**

      Answer:

      Objective:

      Create a C++ program using objects, not functions, that implements a Sudoku puzzle solver. The program must use a recursive, object-oriented approach with depth-first search and backtracking. Each Sudoku puzzle consists of a 9x9 grid divided into nine 3x3 blocks. The program must read five Sudoku puzzle files, find all valid solutions, and display the results.

      Requirements:

      1. The program must read Sudoku puzzles from five text files: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt.
      2. Each puzzle contains a 9x9 grid with digits from 1 to 9 and underscores (_) representing empty cells.
      3. Each Sudoku puzzle must follow these rules:

- Each digit from 1 to 9 must appear exactly once in each row.
- Each digit from 1 to 9 must appear exactly once in each column.
- Each digit from 1 to 9 must appear exactly once in each 3x3 block.

4. The program must use a recursive algorithm with depth-first search and backtracking to solve the puzzles.
5. The program must identify an empty cell and determine which digits from 1 to 9 can be placed in that cell without violating the Sudoku rules.
6. If a digit is valid, the program must place it in the empty cell and recursively attempt to solve the remaining puzzle.
7. If a digit leads to a situation where no valid solution is possible, the program must backtrack by removing that digit and trying another possible digit.
8. If no valid digit can be placed in an empty cell, the program must backtrack to a previous cell and try another possibility.
9. The program must continue searching until all valid solutions are found or all possible combinations have been checked.
10. If a puzzle has more than one valid solution, the program must find and print all valid solutions instead of stopping after the first solution.
11. If a puzzle has no valid solution, the program must display "No solution found".
12. For each puzzle, the program must display:
    - The puzzle file name (e.g., puzzle1.txt).
    - The original puzzle read from the file.
    - All valid solutions found, or "No solution found" if no solution exists.
13. The program must preserve the original numbers provided in each puzzle and only fill cells represented by underscores (_).
14. The program must process all five puzzle files and display the results for each puzzle separately.
15. Use appropriate C++ objects/classes instead of a function-based design to keep the program organized and maintainable.
16. Implement the Sudoku-solving operations using objects/classes, including reading puzzle files, storing the Sudoku grid, checking valid numbers, recursive solving, backtracking, and displaying solutions.
17. Include clear comments explaining the major parts of the code.

d.Generated code from both GenAIs.

Answer:

Gemini:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

// SudokuGame class encapsulates all data and operations for solving a Sudoku puzzle
class SudokuGame {
private:
    std::string filename;
    char originalGrid[9][9];
    char currentGrid[9][9];
    int solutionCount;

    // Helper method to check if placing a digit at a specific cell is valid
    bool isValid(int row, int col, char num) {
        // Check row and column constraints
        for (int i = 0; i < 9; i++) {
            if (currentGrid[row][i] == num) return false;
            if (currentGrid[i][col] == num) return false;
        }
```

```
        // Check 3x3 block constraints
        int startRow = row - (row % 3);
        int startCol = col - (col % 3);
        for (int r = 0; r < 3; r++) {
            for (int c = 0; c < 3; c++) {
                if (currentGrid[startRow + r][startCol + c] == num) return false;
            }
        }

        return true;
    }

    // Recursive depth-first search and backtracking method to find all solutions
    void solve(int row, int col) {
        // Base case: If we have successfully traversed past the last row, a solution is
found
        if (row == 9) {
            solutionCount++;
            std::cout << "Solution " << solutionCount << ":\n";
            displayGrid(currentGrid);
            return;
        }

        // Determine coordinates for the next cell
```

```
    int nextRow = (col == 8) ? row + 1 : row;
    int nextCol = (col == 8) ? 0 : col + 1;

    // If the current cell is pre-filled, skip to the next cell
    if (currentGrid[row][col] != '_') {
        solve(nextRow, nextCol);
        return;
    }

    // Try placing digits from '1' to '9'
    for (char num = '1'; num <= '9'; num++) {
        if (isValid(row, col, num)) {
            currentGrid[row][col] = num; // Place digit
            solve(nextRow, nextCol);     // Recurse to next cell
            currentGrid[row][col] = '_'; // Backtrack by resetting cell
        }
    }
}

// Helper method to print any 9x9 grid
void displayGrid(char grid[9][9]) {
    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            std::cout << grid[i][j] << " ";
```

```
            }
            std::cout << "\n";
        }
        std::cout << "\n";
    }

public:
    // Constructor initializes the filename and solution counter
    SudokuGame(const std::string& fname) : filename(fname), solutionCount(0)
{}

    // Method to read the puzzle grid from the designated text file
    bool loadPuzzle() {
        std::ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }

        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                file >> originalGrid[i][j];
                currentGrid[i][j] = originalGrid[i][j];
            }
        }
```

```
    file.close();
    return true;
}

// Method to manage the solving lifecycle for the puzzle
void process() {
    std::cout << "========================================\n";
    std::cout << "Puzzle File: " << filename << "\n";
    std::cout << "========================================\n";

    if (!loadPuzzle()) {
        std::cout << "Error: Could not open file " << filename << "\n\n";
        return;
    }

    std::cout << "Original Puzzle:\n";
    displayGrid(originalGrid);

    solutionCount = 0;
    solve(0, 0);

    // If no solutions were found, display notification
    if (solutionCount == 0) {
        std::cout << "No solution found.\n\n";
```

```
        } else {
            std::cout << "Total valid solutions found: " << solutionCount << "\n\n";
        }
    }
};

int main() {
    // Array containing the five required puzzle file names
    std::string puzzleFiles[5] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    // Process each puzzle file sequentially using the object-oriented approach
    for (int i = 0; i < 5; i++) {
        SudokuGame game(puzzleFiles[i]);
        game.process();
    }

    return 0;
}
```

Claude:

```
//
================================================================
====================
// Object-Oriented Sudoku Solver
// Recursive depth-first search with backtracking; finds ALL solutions.
//
// Classes:
//   SudokuGrid       - stores the 9x9 board, checks rule validity, prints it
//   PuzzleReader     - reads a puzzle text file into a SudokuGrid
//   SudokuSolver     - recursive DFS/backtracking engine, collects solutions
//   PuzzleReport     - displays file name, original puzzle, and results
//   PuzzleProcessor  - ties everything together for one puzzle file
//   Application      - runs all five puzzle files
//
// Build:  g++ -std=c++17 -O2 -o sudoku sudoku_solver.cpp
// Run:    ./sudoku        (puzzle1.txt ... puzzle5.txt in the working dir)
//
================================================================
====================
#include <array>
#include <fstream>
#include <iostream>
#include <string>
```

```
#include <vector>

// ---------------------------------------------------------------------------
// SudokuGrid: holds a 9x9 board. 0 internally means "empty" (shown as '_').
// ---------------------------------------------------------------------------
class SudokuGrid {
public:
    static const int SIZE = 9;       // 9x9 board
    static const int BLOCK = 3;      // 3x3 blocks
    static const int EMPTY = 0;      // internal marker for an empty cell

    SudokuGrid() {
        for (auto &row : cells) row.fill(EMPTY);
    }

    int  get(int r, int c) const { return cells[r][c]; }
    void set(int r, int c, int v) { cells[r][c] = v; }
    bool isEmpty(int r, int c) const { return cells[r][c] == EMPTY; }

    // Finds the next empty cell scanning row by row.
    // Returns true and fills (r, c) if one exists; false if the grid is full.
    bool findEmptyCell(int &r, int &c) const {
        for (r = 0; r < SIZE; ++r)
            for (c = 0; c < SIZE; ++c)
```

```
            if (isEmpty(r, c)) return true;
    return false;
}

// Checks whether placing `digit` at (r, c) would break the row, column,
// or 3x3 block rule (the cell itself is ignored in the comparison).
bool canPlace(int r, int c, int digit) const {
    // Row and column check
    for (int i = 0; i < SIZE; ++i) {
        if (i != c && cells[r][i] == digit) return false;
        if (i != r && cells[i][c] == digit) return false;
    }
    // 3x3 block check
    int br = (r / BLOCK) * BLOCK, bc = (c / BLOCK) * BLOCK;
    for (int i = br; i < br + BLOCK; ++i)
        for (int j = bc; j < bc + BLOCK; ++j)
            if ((i != r || j != c) && cells[i][j] == digit) return false;
    return true;
}

// Verifies that the digits already on the board do not conflict.
bool isConsistent() const {
    for (int r = 0; r < SIZE; ++r)
        for (int c = 0; c < SIZE; ++c)
```

```
            if (!isEmpty(r, c) && !canPlace(r, c, cells[r][c]))
                return false;
        return true;
    }

    // Prints the grid with '_' for empty cells.
    void print(std::ostream &out) const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                if (isEmpty(r, c)) out << '_';
                else out << cells[r][c];
                if (c < SIZE - 1) out << ' ';
            }
            out << '\n';
        }
    }

private:
    std::array<std::array<int, SIZE>, SIZE> cells;
};

// ---------------------------------------------------------------------
// PuzzleReader: loads a puzzle file into a SudokuGrid.
// Accepts digits 1-9 and '_' for blanks; whitespace is ignored, so both
```

```
// "5_3_" and "5 _ 3 _" layouts work. Needs exactly 81 cell characters.
// ------------------------------------------------------------------------
class PuzzleReader {
public:
    // Returns true on success; on failure `error` describes the problem.
    bool read(const std::string &fileName, SudokuGrid &grid, std::string &error)
const {
        std::ifstream in(fileName);
        if (!in) {
            error = "Could not open file";
            return false;
        }
        int count = 0;
        char ch;
        while (in.get(ch)) {
            if (ch == ' ' || ch == '\t' || ch == '\n' || ch == '\r') continue;
            if (count >= SudokuGrid::SIZE * SudokuGrid::SIZE) {
                error = "File contains more than 81 cells";
                return false;
            }
            int r = count / SudokuGrid::SIZE, c = count % SudokuGrid::SIZE;
            if (ch == '_') {
                grid.set(r, c, SudokuGrid::EMPTY);
            } else if (ch >= '1' && ch <= '9') {
```

```
            grid.set(r, c, ch - '0');
        } else {
            error = std::string("Invalid character '") + ch + "'";
            return false;
        }
        ++count;
    }
    if (count != SudokuGrid::SIZE * SudokuGrid::SIZE) {
        error = "File contains fewer than 81 cells";
        return false;
    }
    return true;
  }
};

// ---------------------------------------------------------------------
// SudokuSolver: recursive depth-first search with backtracking.
// Works on a private copy of the puzzle, so original clues are preserved
// (only empty cells are ever modified). Collects every valid solution.
// ---------------------------------------------------------------------
class SudokuSolver {
public:
  explicit SudokuSolver(const SudokuGrid &puzzle) : working(puzzle) {}
```

```
    // Runs the search and returns all solutions found.
    const std::vector<SudokuGrid> &solveAll() {
        solutions.clear();
        // If the given clues already conflict, there can be no solution.
        if (working.isConsistent()) search();
        return solutions;
    }

private:
    SudokuGrid working;                 // board being filled in / undone
    std::vector<SudokuGrid> solutions;  // every complete valid grid found

    // Recursive DFS step: fill the next empty cell, recurse, then backtrack.
    void search() {
        int r, c;
        // Base case: no empty cell left -> the board is a complete solution.
        if (!working.findEmptyCell(r, c)) {
            solutions.push_back(working);
            return;
        }
        // Try each digit 1-9 in this cell.
        for (int digit = 1; digit <= 9; ++digit) {
            if (working.canPlace(r, c, digit)) {
                working.set(r, c, digit);      // place the digit
```

```
            search();                   // go deeper (DFS)
            working.set(r, c, SudokuGrid::EMPTY); // BACKTRACK: undo and try next digit
        }
    }
    // If no digit fit, the loop ends and we return, which backtracks
    // to the previous cell. We do NOT stop at the first solution, so
    // the search continues until every possibility has been examined.
  }
};

// ---------------------------------------------------------------------
// PuzzleReport: responsible for all output formatting.
// ---------------------------------------------------------------------
class PuzzleReport {
public:
  void showHeader(const std::string &fileName) const {
    std::cout << "==============================\n";
    std::cout << "Puzzle file: " << fileName << "\n";
    std::cout << "==============================\n";
  }

  void showOriginal(const SudokuGrid &grid) const {
    std::cout << "Original puzzle:\n";
```

```
        grid.print(std::cout);
        std::cout << '\n';
    }

    void showSolutions(const std::vector<SudokuGrid> &solutions) const {
        if (solutions.empty()) {
            std::cout << "No solution found\n\n";
            return;
        }
        std::cout << "Solutions found: " << solutions.size() << "\n\n";
        for (size_t i = 0; i < solutions.size(); ++i) {
            std::cout << "Solution " << (i + 1) << ":\n";
            solutions[i].print(std::cout);
            std::cout << '\n';
        }
    }

    void showError(const std::string &message) const {
        std::cout << "Error: " << message << "\n\n";
    }
};

// ---------------------------------------------------------------------
// PuzzleProcessor: handles one puzzle file from reading to reporting.
```

```
// ----------------------------------------------------------------------
class PuzzleProcessor {
public:
    explicit PuzzleProcessor(const std::string &file) : fileName(file) {}

    void process() const {
        PuzzleReport report;
        PuzzleReader reader;
        SudokuGrid puzzle;
        std::string error;

        report.showHeader(fileName);
        if (!reader.read(fileName, puzzle, error)) {
            report.showError(error);
            return;
        }
        report.showOriginal(puzzle);

        SudokuSolver solver(puzzle);
        report.showSolutions(solver.solveAll());
    }

private:
    std::string fileName;
```

```
};

// ------------------------------------------------------------------------
// Application: runs the solver on all five puzzle files.
// ------------------------------------------------------------------------
class Application {
public:
    void run() const {
        const std::vector<std::string> files = {
            "puzzle1.txt", "puzzle2.txt", "puzzle3.txt",
            "puzzle4.txt", "puzzle5.txt"};
        for (const auto &f : files) {
            PuzzleProcessor processor(f);
            processor.process();
        }
    }
};

// Program entry point: just creates and runs the Application object.
int main() {
    Application app;
    app.run();
    return 0;
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the "GenAI Code Comparison" lecture.

Answer:

Correctness: Both Gemini and Claude models generates C++ code implementing the idea of using recursion/backtracking to solve the Sudoku puzzle. Both programs correctly check rows, columns, and blocks of 3×3 grids before writing down numbers and have an option to choose multiple solutions. However, Claude is better at providing a correct solution as it checks for invalid characters in the input file, missing numbers in the puzzle file, or conflicting numbers in the original puzzle. On the other hand, Gemini does not have these mechanisms, making it possible to select invalid puzzle files.

Execution Time: Both programs make use of the backtracking method, which could be time-consuming for puzzles with many empty cells. I believe that Gemini performs faster than Claude because it moves directly to the next empty cell, while the second program has to scan the Sudoku board to find an empty cell each time. Both programs prevent illegal numbers by following the same algorithm and immediately returning if a number does not meet the requirements, thus eliminating the need to check all possibilities. Further performance evaluation has to be done to see which program is actually faster.

Space Complexity: Gemini makes use of two 9x9 matrices to store the original and the current puzzle. In addition, it uses memory to save the recursive calls, however, it does not save the solutions, it immediately prints them out. On the other hand, Claude uses one 9x9 matrix and recursion, but it saves all solutions in a vector before displaying. Thus, Gemini utilizes lesser memory, especially when there are multiple solutions to a given puzzle.

Maintainability: Gemini utilizes one SudokuGame class to manage the file-reading, number-checking, puzzle-solving functions, and display of results. This

decreases the code's size and complexity, but may lead to complications if any alterations are made (since every part of the code is connected to one class). On the contrary, Claude's program makes use of six different classes, where each manages one particular set of tasks. This allows for easier editing and arrangement of the code, but leads to a more complex and voluminous code in comparison to Gemini's generated code.

f. Select one to use as the basis for this assignment and justify why you picked it.

Answer: I chose Gemini's code for this assignment because it is simpler, more understandable, and less memory-intensive. Gemini's code uses fewer classes - only one SudokuGame class that contains all the logic operations. Also, it moves to the next cell directly, instead of searching for it every time, which can make the code run faster. Finally, Gemini does not save all the solutions in a vector, like Claude does - thus, being more memory-efficient. Although Claude's code correctly validates user input, such validation can be added to Gemini's code as well. Thus, Gemini is a better option for this assignment, as it is simpler and more efficient.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Answer:

Correctness: I have improved the code that is generated by Gemini, implementing several checks for invalid characters, missing or extra numbers in the puzzle file, and by adding a check for initial invalidity of the puzzle (inconsistency in any row, column, or block). These improvements allow the program to handle invalid puzzles by detecting and reporting errors instead of providing wrong solutions.

Execution Time: I kept Gemini's strategy of moving right to the next cell because that way you don't have to keep checking the entire board. I also added a

check to see if the original puzzle has any conflicts or not. This helps to limit the amount of work to do after putting in numbers.

Space Complexity: The two fixed 9x9 arrays used by Gemini to store the original and the current puzzles are retained. As well as the method of printing the solution immediately when found instead of saving them in a vector to print them all at once is kept too. This allows the program to have minimal memory consumption and still display all possible solutions.

Maintainability: I have enhanced the code of Gemini through providing more informative comments on the main operations, including file reading, number checking, recursion, and backtracking, as well as improving error messages. As a result, the program became easier to read, maintain, and update.